

Enhancing Event-Frame Feature Tracking with Siamese FPN

Andreas Fotakis[✉] and Emmanouil Psarakis[✉]

Department of Computer Engineering and Informatics, University of Patras, 26504
Rion-Patras, Greece
{st1084674,psarakis}@ceid.upatras.gr

Abstract. In this paper modifications to the pipeline of the first data-driven model based feature tracker for event cameras, are proposed, that improve robustness and accuracy under domain shift. Specifically, the replacement of the separate Feature Pyramid Networks (FPN) for the event and frame branches, with a shared-weight Siamese architecture that learns a common multi-scale descriptor for both modalities, resulting in tighter cross modal alignment in a shared feature space and drastically reducing redundant parameters. Furthermore, a RAFT-style loss is adopted to supervise intermediate displacement predictions and promote more stable optimization. Experimental results show that the proposed method generalizes effectively across datasets and achieves a significant improvement over the initial zero-shot transfer performance from synthetic to real data on the established event-based feature tracking benchmark. The code is available at: <https://github.com/afotakis/FESiamTrack>.

Keywords: Event Camera · Feature Tracking · Siamese Network

1 Introduction

Event cameras have attracted increasing attention in computer vision because they measure brightness changes asynchronously, with microsecond temporal resolution, high dynamic range, and low latency. Instead of producing dense frames at a fixed rate, an event camera outputs a stream of events whenever the log-intensity at a pixel changes by a predefined threshold. These properties make event cameras particularly suitable for motion-centric tasks such as visual odometry, SLAM, and feature tracking, especially in scenarios involving fast camera motion, motion blur, or challenging illumination conditions.

Feature tracking is a cornerstone of geometric vision pipelines, providing the essential temporal correspondences required for tasks like motion estimation, pose tracking, and structure reconstruction. While well-established for conventional cameras, robust feature tracking remains a persistent challenge for event-based sensors. Events are inherently sparse, asynchronous, and dependent on local texture and motion dynamics, completely lacking the absolute appearance

data found in standard frames. Consequently, establishing reliable associations between an initialized feature and subsequent event streams is highly difficult. This challenge is further exacerbated by the severe domain shift encountered when deploying models trained on synthetic data to real-world sequences, where sensor noise and differing scene statistics often degrade tracking reliability.

Classical event-based feature trackers are typically based on hand-crafted model assumptions, such as iterative optimization of motion parameters [6, 13, 26], probabilistic event-to-feature data association [26], continuous trajectory or curve-based motion parameterization [3, 21], or discrete hypothesis selection over candidate feature translations [1]. Although these methods can exploit the high temporal resolution of event cameras to produce accurate and low-latency feature tracks, their performance can degrade in the presence of noise and unmodeled effects, and they often require careful parameter tuning. More recently, learning-based approaches have shown that event-based feature tracking can be formulated as a data-driven matching problem. In particular, DDFT [19] introduced a learned event-frame tracker in which a reference frame patch and an event-based search patch are encoded by separate Feature Pyramid Networks and matched through a correlation-based module followed by temporal refinement. This formulation demonstrated the potential of supervised learning for event-based feature tracking, but the use of separate modality-specific encoders also introduces redundant parameters and does not explicitly enforce a shared representation between the frame and event branches.

The idea of learning a shared matching space is strongly motivated by the broader visual tracking literature, where Siamese architectures have been extensively used to compare a target or reference template with a search region through shared-weight feature encoders and correlation-based similarity matching [2, 14, 15, 25].

To improve robustness under synthetic-to-real domain shift, this paper proposes a modified event-frame feature tracking pipeline utilizing a shared-weight Siamese Feature Pyramid Encoder (FPE). By embedding reference frame patches and event representations into a common descriptor space, this approach constrains matching and aligns modalities across scales while reducing redundant parameters. In addition to the Siamese FPN, we adopt a RAFT-style loss for training the unrolled tracker. Instead of supervising only the final or truncated displacement prediction, the proposed loss supervises all intermediate predictions produced during unrolling, with exponentially increasing importance assigned to later predictions. This encourages the recurrent tracker to refine its displacement estimates over time while still providing direct supervision to earlier updates. Although this loss does not substantially improve performance on the synthetic test set, our experiments show that it improves generalization to real event-camera datasets, suggesting that sequence-level supervision is particularly useful under real-world domain shift.

The proposed pipeline is trained on synthetic MultiFlow sequences and using the evaluation protocol established in prior event-based feature tracking work on

real event-camera datasets with the established feature age and expected feature age metrics.

The contributions of this paper can be summarized as follows:

- We propose a Siamese shared-weight Feature Pyramid Network for event-frame feature tracking, which maps frame patches and event tensors into a common multi-scale descriptor space while reducing redundant parameters and improving performance.
- We adopt a RAFT-style sequence loss that supervises all intermediate unrolled displacement predictions and improves robustness under synthetic-to-real domain shift.

The rest of the paper is structured as follows: Section 2 presents the prior work. In Section 3 we formulate the problem and present the proposed methodology. In Section 4 we present the architecture of the proposed pipeline which is followed by the experimental results in Section 5. Finally, Section 6 concludes this paper.

2 Related Work

Classical feature tracking methods with events. Before learning-based approaches, event-camera feature tracking was dominated by model-driven methods that explicitly align the recent event stream to a feature model via iterative optimization or event-by-event multi hypothesis testing. ICP based trackers represent each feature as a local set of recent events and estimate its motion by repeatedly associating incoming events to a reference point set and minimizing a (weighted) point-to-point alignment error, enabling low-latency feature tracks that can be used in downstream VO pipelines [13]. In [26] an EM-ICP formulation is proposed that couples ICP alignment with *probabilistic data association*. More specifically, an intertwined expectation-maximization objective soft-assigns events to candidate tracks while optimizing optical flow (and optionally affine deformation/persistence), yielding longer and more stable tracks under ambiguous event observations. Complementary to iterative alignment, HASTE [1] performs *event-driven* tracking via multi-hypothesis testing. Specifically, the motion space is discretized into a number of candidate rotations/translations and updates hypothesis alignment scores per incoming event, triggering state transitions once confidence thresholds are exceeded, which improves efficiency while retaining competitive tracking quality. Finally, EKLT [6] exploits the complementarity of DAVIS-style sensors by initializing features on frames and then tracking them asynchronously using events within a maximum-likelihood framework based on a generative event model, optimizing a parametric warp (and velocity) to achieve subpixel, long-lived feature tracks.

Learning-based feature tracking methods with events. A major shift from hand-crafted feature trackers is the use of neural patch-localization pipelines

that learn to associate an event-derived search patch with a fixed visual template. Recently, the first learning-based tracker was introduced for event cameras [19], which leverages low-latency events to track features initialized in a grayscale frames. To this end, the use of separate multi-scale FPN [17] backbones for the encoding of event tensors and frame templates followed by a correlation-based matching stage that localizes the template within the current event patch, with temporal refinement to stabilize the track, is proposed. Building on the same encode-then-match paradigm, FE-TAP [18] similarly relies on FPN encoders to extract multi-scale descriptors from event and frame inputs before performing learned matching and iterative refinement to maintain point trajectories under high-speed motion. Extending learned tracking to 3D, E-3DTrack [16] preserves the same FPN-centric front-end for multi-scale feature extraction and correlation-driven alignment, but augments it with stereo-based depth estimation so that the matched 2D tracks can be lifted to 3D trajectories. Beyond these hybrid event-frame pipelines, in [19] also show that the same learned tracking formulation can be deployed in an event-only configuration (i.e., operating purely on events after initialization), enabling tracking in regimes where frames are unreliable due to blur or low-light. More recent work pushes this direction further by moving from sparse keypoints to event-only tracking-any-point: ETAP [8] introduces an event-only point-tracking framework trained to produce long-range semi-dense trajectories directly from events, explicitly targeting robustness under high-speed motion and challenging illumination conditions. Similarly, MATE [9] proposes an event-only tracker that injects motion cues into the matching process (via a motion-guidance/kinematics signal) and enforces temporal consistency during iterative refinement, improving stability under event sparsity and motion sensitivity.

Correlation and Siamese matching as a tracking primitive. A common and highly effective tracking paradigm is to cast tracking as offline-learned matching between a fixed template and a search region, where the online problem reduces to evaluating a learned similarity function. Early Siamese trackers such as SINT [23] learn a generic matching function that compares the initial target patch with candidate patches in subsequent frames, avoiding online model adaptation. SiamFC [2] further formulates this idea in a fully convolutional manner, localizing the target by computing a dense response map through cross-correlation between template and search features, enabling fast tracking without expensive online optimization. Subsequent works enhance this correlation-based formulation by attaching task heads that jointly perform classification and geometric regression, e.g., SiamRPN [15] and its deeper variants such as SiamRPN++ [14], while preserving the core template-search matching primitive. Closely related ideas appear in dense correspondence. FlowNet popularized explicit feature correlation for learned optical-flow matching [5], PWC-Net builds cost volumes within a pyramid-and-warping scheme [22], and RAFT constructs all-pairs correlation volumes and iteratively refines correspondences with a recurrent update operator [24]. Recent point tracking methods

can be interpreted as sparse, long-range instantiations of the same correspondence principle: PIPs combines cost volumes with iterative inference to estimate point trajectories through occlusions [10], TAPIR performs per-frame matching followed by temporal refinement driven by local correlations [4], and CoTracker improves robustness by tracking many points jointly with a transformer that exploits dependencies between trajectories [12].

3 Problem Formulation

3.1 Preliminaries

Let us define the sets $\mathcal{N}_i = \{0, 1, 2, \dots, S_i - 1\}$, $i = 1, 2$ with S_i , $i = 1, 2$ denoting the height (H) and width (W) of a full image $I(\mathbf{n})$ and their cartesian product:

$$\mathcal{S}_I = \mathcal{N}_1 \times \mathcal{N}_2 = \left\{ \mathbf{n} \in \mathbb{Z}^2 \mid n_i \in \mathcal{N}_i, i = 1, 2 \right\} \quad (1)$$

which constitutes the support of the full image¹, and let its continuous counterpart be defined as $\mathcal{X}_{\mathcal{S}_I} = \left\{ \mathbf{x} \in \mathbb{R}^2 \mid 0 \leq x_i \leq S_i - 1, i = 1, 2 \right\}$.

The event-generation mechanism is described by the change in logarithmic image intensity: $\Delta L(\mathbf{x}(t), t) = L(\mathbf{x}(t), t) - L(\mathbf{x}(t), t - \Delta t)$ or, equivalently,

$$\Delta L(\mathbf{x}(t), t) = \ln \left(\frac{I(\mathbf{x}(t), t)}{I(\mathbf{x}(t), t - \Delta t)} \right). \quad (2)$$

An event is generated at time t_k when the magnitude of the log-intensity change reaches a predefined contrast threshold C , $|\Delta L(\mathbf{x}(t_k), t_k)| \geq C$.

The resulting event is represented as $\mathbf{e}_k \doteq [\mathbf{n}_k^t, t_k, p_k]^t$ at pixel $\mathbf{n}_k = \lfloor \mathbf{x}(t_k) + 1/2 \rfloor$, where $\lfloor \cdot \rfloor$ denotes the floor function and the polarity $p_k = \pm 1$ indicates the sign of the intensity change.

3.2 The proposed solution

The training of a feature tracker remains challenging because, in general, datasets with dense pixel-level correspondences over time are scarce. This limitation is more pronounced for event-based data where accurate correspondence annotations are rarely available. We therefore, by following the strategy proposed in [19], train our network using synthetic sequences. Since for the synthetic data, ground-truth feature tracks are available, and that makes the network's training an easier problem.

Let $\mathcal{I} = \{I_s(\mathbf{n})\}_{s=1}^{N_s}$ be the set of the first frames of the available synthetic sequences of the training set and $\mathcal{F} = \{f_l\}_{l=1}^{N_f}$ be the set of all the features detected in the above mentioned set of images. Then, for a feature f_l belonging

¹ The support of each event patch denoted by $\mathcal{S}_P$ can be define in a similar way.

in the synthetic image $I_s(\mathbf{n})$ of set $\mathcal{I}$ we can define its neighborhood $\mathcal{N}_{f_l}(t_j)$ of size $(2N+1) \times (2N+1)$ by:

$$\mathcal{N}_{f_l}(t_j) = \left\{ \mathbf{x}(t_j) \in \mathcal{X}_{\mathcal{S}_{I_s}} \mid \|\mathbf{x}(t_j) - \mathbf{x}_{f_l}(t_j)\|_p \leq N \right\} \quad (3)$$

with respect to the global coordinate system with $\mathbf{x}_{f_l}(t_j)$, $\|\mathbf{z}\|_p$ denoting its position in $\mathcal{X}_{\mathcal{S}_{I_s}}$ at that time instance and the l_p norm of vector $\mathbf{z}$ respectively. It is clear that $\mathcal{N}_{f_l}(t_0)$ is of special form, derived from Equation (3) but with $\mathbf{x}_{f_l}(t_0) \in \mathcal{S}_{I_s}$ defined by Equation (1). For the feature point f_l in the reference frame $I_s(\mathbf{n})$ we extract a reference patch denoted by P_l . Note that its support coincides with the neighborhood $\mathcal{N}_{f_l}(t_0)$ of the specific feature point.

Our goal now is to estimate its continuous trajectory $\mathbf{x}_{f_l}(t)$ for the time interval $t_0 \leq t \leq T_l$ where $T_l - t_0$ denoting the trajectory duration of the feature f_l . To this end we are going to formulate the tracking problem over progressively increasing intervals T_q , $q = \{1, 2, 3\}$, $T_q = M_q T$, with $M_q = \{4, 16, 24\}$ denoting the corresponding unrolling size and T the duration of the fundamental time window. For such an interval T_q starting at time instance t_{j-1} , let us define the following events' set:

$$\mathcal{E}_q = \{\mathbf{e}_k | t_k \in T_q\}, \quad (4)$$

containing the stream of events generated in the temporal segment T_q . For each feature f_l , the events in $\mathcal{E}_q$ are spatially cropped around its current location and converted into the SBT-Max representation [19], yielding the local event tensor $V_{j,l}$ used as input to the tracker. The complete construction of this representation is provided in the Supplementary material.

Let $\Delta \mathbf{x}_{f_l}^{GT}(t_j)$ denote the ground-truth displacement of feature f_l from $\mathbf{x}_{f_l}(t_{j-1})$ to $\mathbf{x}_{f_l}(t_j)$ over the fundamental tracking interval $T \ll T_q$, where:

$$t_j = t_{j-1} + T. \quad (5)$$

Let Θ denote the parameters of the tracker, as detailed in Section 4, and let $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta)$ denote its predicted displacement. The optimal model parameters are obtained by solving:

$$\Theta^* = \arg \min_{\Theta} L(\Theta), \quad (6)$$

where $L(\cdot)$ the loss function, defined in Section 4.3, that measures the discrepancy between the ground-truth and predicted displacements, $\Delta \mathbf{x}_{f_l}^{GT}(t_j)$ and $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta)$, respectively.

This procedure is repeated for all time intervals T_q starting for the shortest one. By using the following updated rule:

$$\hat{\mathbf{x}}_{f_l}(t_j; \Theta^*) = \hat{\mathbf{x}}_{f_l}(t_{j-1}; \Theta^*) + \Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta^*), \quad (7)$$

the tracker yields a temporally ordered set of estimates $\{\hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)\}_{j=1}^M$ and, by interpolation if desired, a dense trajectory.

Finally we must stress at this point that we keep the reference patch P_l fixed throughout tracking; namely after each update in Equation (7), we extract the next local event stream of duration T centered at the newly predicted location $\hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)$, while matching against the same reference appearance P_l .

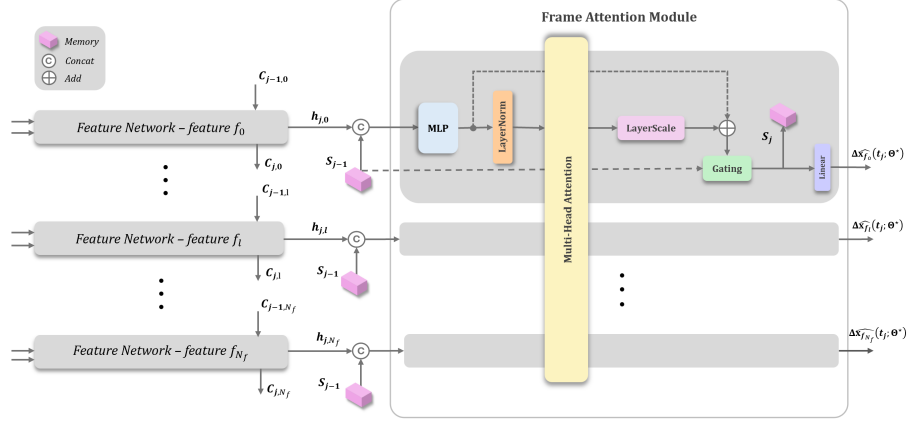


Fig. 1: Overview of the tracking pipeline for feature f_l at step j . The reference patch P_l and event tensor $V_{j,l}$ are processed by the Feature Network in Figure 2, which matches their features and refines the result with a ConvLSTM to produce hidden state $\mathbf{h}_{j,l}$ and cell output $\mathbf{c}_{j,l}$. The Frame Attention Module combines $\mathbf{h}_{j,l}$ with the previous track state $\mathbf{s}_{j-1,l}$, aggregates information across tracks, and updates $\mathbf{s}_{j,l}$. A linear head then predicts the incremental displacement $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)$, which is accumulated using Equation (7).

4 Architecture

As it was mentioned in previous section, the proposed tracker is similar to that proposed in [19] and is shown in Figure 1. Thus, each reference appearance patch P_l , $l = 1, 2, \dots, N_f$ is encoded once and kept fixed, while each subsequent interval T provides an event tensor centered at the current estimate $\hat{\mathbf{x}}_{f_l}(t_{j-1}; \Theta^*)$. Given these inputs, the network predicts the incremental displacement $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)$ defined in Equation (7). Specifically, the main parts of the proposed pipeline which is shown in Figure 1, are:

1. A Feature Network which is shown in Figure 2 composed by:
 - A Siamese FPE, which is responsible for the extraction of features from the two different input modalities and for their mapping onto a common latent space.
 - An event Feature Pyramid Decoder (FPD) which is combined with the event Siamese FPE thus forming a FPN, which is responsible for the construction of the events feature map.
 - A Correlation map between the current event features and the cached reference descriptor is computed, and
 - a ConvLSTM that exploits a probability map, resulting from the correlation of the events feature tensor and the frame latent embedding, and the events feature tensor for modeling the spatiotemporal dependencies.
2. A Frame Attention Module (FAM) proposed in [19] which captures temporal as well as spatial correlations between tracks within the same frame.

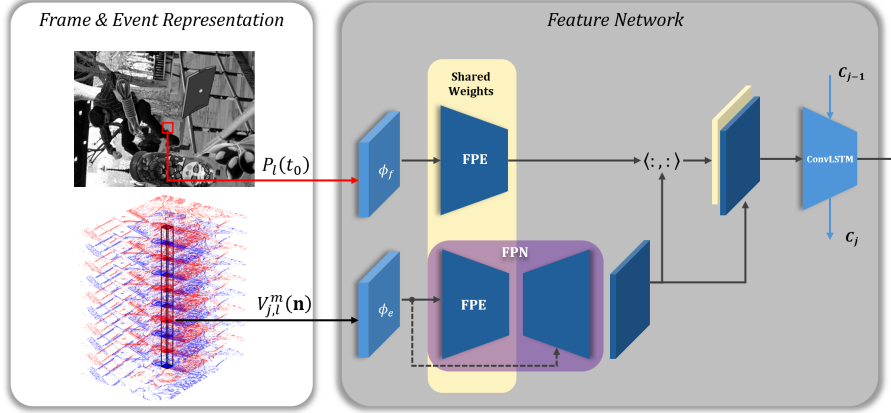


Fig. 2: Detailed architecture of the Feature Network. Modality-specific stems ϕ_f and ϕ_e process the reference patch P_l and event tensor $V_{j,l}$ before a shared-weight Feature Pyramid Encoder. The frame branch produces the cached reference descriptor $\mathbf{l}_l^f$, while the event branch is decoded into the spatial feature map $D_{j,l}^e$. Correlating $D_{j,l}^e$ with $\mathbf{l}_l^f$ yields a correlation map $C_{j,l}$, which are converted by spatial softmax into the matching-probability map $\mathbb{P}_{j,l}$. This map is concatenated with $D_{j,l}^e$, processed by two convolutional blocks, and passed to the ConvLSTM, which outputs the refined representation $\mathbf{h}_{j,l}$ and updated state $\mathbf{c}_{j,l}$.

4.1 Siamese Shared Encoder-weights FPN

A Siamese Neural Network is a deep learning architecture that uses two identical subnetworks, sharing the same parameters and weights, to process distinct inputs in parallel. The high-level feature vectors produced by these networks, that is its embeddings, the model learns to determine the similarity between the inputs of different modality. Inspired by that, we encode events and frames via a siamese network, that share common FPE parameters θ^e , i.e.,:

$$\mathbf{l}_l^f = \Phi_{\text{FPE}}(\phi_f(P_l(t_0); \theta_0^f); \theta^e), \quad (8)$$

$$\{\mathbf{l}_{j,l}^e, D_{j,l}^e\} = \Phi_{\text{FPN}}(\phi_e(V_{j,l}^m(\mathbf{n}); \theta_0^e); \theta^e, \theta^d), \quad (9)$$

where $\phi_f(\cdot; \theta_0^f)$ and $\phi_e(\cdot; \theta_0^e)$ are lightweight modality-specific input stems producing 32 channels with θ_0^f , θ_0^e denoting their learnable parameters, $\mathbf{l}_l^f$, $\mathbf{l}_{j,l}^e$ the encodings of the Siamese encoder onto the latent space $\mathbb{R}^C$, and an event FPN [17] with θ^d denoting the learnable parameters of its decoder. The event FPN consists of the shared FPE producing a hierarchy of features and a FPD which consists of a top-down pathway with lateral connections and de-aliasing convolutions for reconstructing a high-resolution feature map. The output of the stem $\phi_e(\cdot; \theta_0^e)$ is laterally connected with the last layer of the top-down pathway, as shown in Figure 2. The output is a tensor $D_{j,l}^e \in \mathbb{R}^{C \times H \times W}$.

This approach substantially reduces the memory footprint of the DNN while preserving the inference computational cost as shown in the Supplementary material. In particular, it halves the number of encoder parameters, while requiring only the event-channel decoder, reducing the model size from approximately 35M parameters to about 20M parameters.

The fusion is inherently *matching-driven*: at each temporal interval we correlate the current event-derived feature tensor $D_{j,l}^e$ with the latent encoding $\mathbf{l}_l^f$ of the patch P_l . Specifically, the following correlation map:

$$C_{j,l}(\mathbf{n}) = \langle D_{j,l}^e(:, \mathbf{n}), \mathbf{l}_l^f \rangle, \mathbf{n} \in \mathcal{S}_P$$

and then is converted into a probabilistic matching distribution $\mathbb{P}_{j,l}$ with the use of the spatial softmax function, where $\mathbb{P}_{j,l}(\mathbf{n})$ is the likelihood the tracked feature f_l lies at location $\mathbf{n}$ within the search time window. The above mentioned probability map $\mathbb{P}_j$, concatenated with the feature tensor D_j^e , passes through two ConvBlocks and constitutes the input to the ConvLSTM whose outputs, hidden state $\mathbf{h}_{j,l}$ and cell $\mathbf{c}_{j,l} \in \mathbb{R}^{256}$ feed the FAM and the ConvLSTM, of the next time window T , respectively.

We must stress at this point that, the size of the above mentioned ConvBlock layers is modified. Specifically, their sizes are $128 \times \frac{H-1}{2} \times \frac{W-1}{2}$ and $128 \times \frac{H-3}{4} \times \frac{W-3}{4}$ instead of $64 \times \frac{H-1}{2} \times \frac{W-1}{2}$ and $128 \times \frac{H-3}{4} \times \frac{W-3}{4}$ of the DDFT [19].

4.2 Frame Attention Module

This module, proposed in DDFT [19] serves two complementary purposes. First, the attention term captures correlations *between tracks* within the same frame, which is beneficial when multiple tracked patches share motion patterns. Second, the gated memory $\mathbf{s}_{j,l}$ provides a stable per-track summary that smooths updates and mitigates oscillations; namely, when the gate $\mathbf{g}_{j,l}$ is high, the model preserves the previous memory while when it is low, it trusts the newly attended representation. We propose, modifications to this module, to improve stability and performance. Specifically, we propose an additional layer normalization step as well as the dropout operation after attention.

After the ConvLSTM and convolutional processing, a per-track embedding $\mathbf{h}_{j,l} \in \mathbb{R}^{256}$ is obtained. A per-track state $\mathbf{s}_{j-1,l} \in \mathbb{R}^{256}$ is also maintained. Let $\mathbf{z}_{j,l}$ denote the output of the MLP for each feature f_l inside the batch $\mathcal{B}$ of size $N_{\mathcal{B}}$, that is:

$$\mathbf{z}_{j,l} = \text{MLP}([\mathbf{h}_{j,l}, \mathbf{s}_{j-1,l}]) \in \mathbb{R}^{256}. \quad (10)$$

To better stabilize the behavior of the multi-head self-attention mechanism during training, we introduce a pre-normalization step before attention is computed across tracks, i.e.,

$$A_j = \text{MHA}(\text{LayerNorm}(Z_j)), \quad (11)$$

where $Z_j = [\mathbf{z}_{j,1} \ \mathbf{z}_{j,2} \ \cdots \ \mathbf{z}_{j,N_{\mathcal{B}}}] \in \mathbb{R}^{N_{\mathcal{B}} \times 256}$ and $A_j \in \mathbb{R}^{N_{\mathcal{B}} \times 256}$ contains the aggregated information from the other tracks of the same frame, obtained by the MHA module.

We further regularize the attention output with dropout and incorporate it through a residual connection with LayerScale:

$$X_j = Z_j + \text{LayerScale}\left(\text{Dropout}(A_j)\right) \in \mathbb{R}^{N_B \times 256}. \quad (12)$$

Finally, the persistent per-track state is updated using a gated mechanism, that is:

$$\mathbf{g}_{j,l} = \sigma\left(\text{Linear}\left([\mathbf{s}_{j-1,l}; \mathbf{x}_{j,l}]\right)\right) \in \mathbb{R}^{256}, \quad \mathbf{s}_{j,l} = \mathbf{s}_{j-1,l} \odot \mathbf{g}_{j,l} + \mathbf{x}_{j,l} \odot (\mathbf{1} - \mathbf{g}_{j,l}) \quad (13)$$

where $\sigma(\cdot)$ the logistic function, $\mathbf{x}_{j,l}$ is the l -th column of matrix X_j defined in Equation (12), $\mathbf{1}$ the all ones vector of size 256 and $\odot$ denotes the Hadamard operator.

The outputs of the frame-attention module are then linearly regressed to 2D displacements $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)$, $l = 1, \dots, N_B$. Note that in order to improve the ability of the DNN for generalizing on unseen data, we adopt the augmentation strategy proposed in DDFT [19] to $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta^*)$. These displacements are accumulated according to Equation (7) for each feature, the next event patches are centered around $\hat{\mathbf{x}}_{f_l}(t_j; \Theta)$ and the new event tensors are formed, while the reference frames $P_l, l = 1, \dots, N_B$ remain fixed.

4.3 The Proposed Loss function

Let $\Delta \mathbf{x}_{f_l}^{GT}(t_j)$ be the ground-truth displacement from $\mathbf{x}_{f_l}^{GT}(t_{j-1})$ to $\mathbf{x}_{f_l}^{GT}(t_j)$ for the interval $(t_{j-1}, t_j]$, and let $\Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta)$ be the model prediction (cf. Equation (7)). Then we can define the following l_1 norm of the track error:

$$\varepsilon_l^{(j)}(t_j, \Theta) = \left\| \Delta \mathbf{x}_{f_l}^{GT}(t_j) - \Delta \hat{\mathbf{x}}_{f_l}(t_j; \Theta) \right\|_1. \quad (14)$$

Following the RAFT training formulation, each track error in the unrolled sequence is not weighted equally. Instead, in the later ones are assigned larger weights, since they correspond to refined estimates after more iterations of the recurrent update mechanism. Therefore, the weight assigned to the track error at the j -th unrolling step is defined as follows: $w_j = \gamma^{M-j}$, with decay factor $\gamma \in (0, 1)$. Consequently, the terminal track error at $j = M$ retains full weight ($w_M = 1$), whereas earlier errors are exponentially downweighted. Using Equation (14), the RAFT-style sequence loss for feature f_l is defined as:

$$c_{\text{RAFT}}(l, \Theta) = \sum_{j=1}^M \gamma^{M-j} \varepsilon_l^{(j)}(t_j, \Theta) = \sum_{j=1}^M \gamma^{M-j} \left\| \Delta \mathbf{x}_{f_l}^{GT}(t_j) - \Delta \hat{\mathbf{x}}_{f_l}^{(j)}(t_j; \Theta) \right\|_1.$$

Finally, averaging over all supervised features in the mini-batch, the total RAFT-style displacement sequence loss is defined as $\mathcal{L}_{\text{RAFT}}(\Theta) = \frac{1}{N_B} \sum_{l=1}^{N_B} c_{\text{RAFT}}(l, \Theta)$, or equivalently:

$$\mathcal{L}_{\text{RAFT}}(\Theta) = \frac{1}{N_B} \sum_{l=1}^{N_B} \sum_{j=1}^M \gamma^{M-j} \left\| \Delta \mathbf{x}_{f_l}^{GT}(t_j) - \Delta \hat{\mathbf{x}}_{f_l}^{(j)}(t_j; \Theta) \right\|_1. \quad (15)$$

We must stress at this point that in contrast to the truncated displacement loss used in [19], this formulation supervises all intermediate predictions produced by the unrolled tracker, even when the tracking error exceeds the search-patch radius. The exponential weighting encourages the network to progressively refine its displacement estimate, while still providing direct supervision to the earlier predictions. In our implementation, the decay factor is initially set to $\gamma = 0.9$. As we are going to see in the next section, although evaluating the pipeline on synthetic data leaves the impact of the proposed loss invariant, its positive effect is evident during its testing in real-world data.

5 Experiments

In this section, we evaluate the performance of the proposed pipeline in terms of its robustness and accuracy under domain shift and compare it against the first data-driven model based feature tracker for event cameras [19]. To ensure a fair comparison, all experimental settings are kept the same as in the baseline method. In particular, we use the same feature initialization procedure, the same number of tracked features, the same event representation, and the same evaluation protocol and scripts. As we are going to see from the obtained experimental results, the proposed method generalizes effectively across datasets and achieves a significant improvement over the initial zero-shot transfer performance from synthetic to real data on the established event-based feature tracking benchmark.

5.1 Datasets and Metrics

Datasets. The proposed pipeline was trained on synthetic data from MultiFlow [7]. MultiFlow provides rendered image sequences, synthetically generated events, and dense ground-truth motion trajectories, making it suitable for supervised training of feature displacement prediction. In our setting, sparse feature tracks are extracted from the synthetic sequences and used to supervise the network over multiple unrolled prediction steps. The synthetic data are adapted to real event statistics, by adopting the on-the-fly geometric augmentation proposed in [19]. On the other hand, the performance of the proposed pipeline was evaluated on the Event Camera dataset (EC) [20] and the Event-aided Direct Sparse Odometry dataset (EDS) [11], using the evaluation protocol established in prior event-based feature tracking work [8, 18, 19]. EC dataset provides APS frames at 24 Hz and events at 240×180 resolution recorded with a DAVIS sensor, together with ground-truth camera poses at 200 Hz from an external motion-capture system, while EDS provides higher-resolution synchronized frames and events at 640×480 and ground-truth camera poses at 150 Hz. Both datasets predominantly contain static scenes, they are nevertheless suitable for benchmarking feature tracking because camera motion induces challenging apparent motion, motion blur in frames, and high event rates.

Table 1: Ablation study of the training loss and the RAFT forgetting factor γ . The proposed pipeline is trained on the synthetic MultiFlow dataset and evaluated on the MultiFlow, EDS, and EC datasets. FA and EFA are reported, with higher values indicating better performance.

	MultiFlow		EDS		EC	
	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$
Loss proposed in [19]	0.845	0.835	0.604	0.515	0.854	0.843
RAFT loss ($\gamma = 0.75$)	0.842	0.832	0.626	0.531	0.830	0.817
RAFT loss ($\gamma = 0.80$)	0.840	0.830	0.616	0.522	0.850	0.837
RAFT loss ($\gamma = 0.90$)	0.843	0.835	0.616	0.521	0.866	0.858
RAFT loss ($\gamma = 0.95$)	0.835	0.823	0.611	0.521	0.863	0.850

Metrics. We evaluate our method by following standard feature tracking protocols. Specifically, in order to evaluate the proposed method and compare its performance against its rivals we are going to well established metrics; namely *Feature Age* (FA) and *Expected Feature Age* (EFA) which quantify the duration until a track deviates beyond a threshold distance from the Ground Truth. Details on the evaluation metrics are provided in Supplementary Material Section 2 and [19].

5.2 Implementation Details

We train on 30K feature tracks using the Adam optimizer with an initial learning rate of 10^{-4} , and gradually increase the unrolling horizon to enable long-term tracking supervision (continual unrolling). Specifically, we start from 4 unrolling steps and progressively increase them to 16 and 24 after 80K and 120K training iteration steps, respectively. We set, the duration of the fundamental tracking interval in Equation (5) at $T = 20$ ms.

5.3 Experimental Results

The Impact of RAFT-style loss. Let us begin by evaluating the impact of the proposed RAFT-style loss defined in Equation (15) on the performance of the pipeline. Table 1 contains the results obtained from the evaluation of the pipeline in terms of its performances on the FA and EFA metrics when the model was trained with the loss proposed in [19] and the RAFT-style loss.

As it is clear from the contents of this table, while the performance of the pipeline seems to be invariant on the synthetic test set, on the real datasets the RAFT-style loss improves its performance substantially. These results demonstrate that the RAFT-style loss is mainly advantageous for generalizing to real-world datasets, where monitoring the sequence of unrolled displacement predictions ensures robust tracking. Furthermore, the effect of the RAFT forgetting factor γ is also examined in Table 1. Across the tested values of γ , lower values

Table 2: Performance of different event-based feature tracking methods on the EDS and EC datasets. Higher values indicate better performance. Best results are highlighted separately for event-frame (E+F) and event-only (E) methods.

Method	Input	EDS		EC	
		FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$
ICP [13]	E+F	0.060	0.040	0.256	0.245
EM-ICP [26]	E	0.161	0.120	0.337	0.334
HASTE [1]	E	0.096	0.063	0.442	0.427
EKLT [6]	E+F	0.325	0.205	0.811	0.775
DDFT (zero-shot) [19]	E+F	0.549	0.451	0.795	0.787
DDFT (fine-tuned) [19]	E+F	0.576	0.472	0.825	0.818
FE-TAP [18]	E+F	0.676	0.589	0.844	0.838
Ours ($\gamma = 0.9$)	E+F	0.616	0.521	0.866	0.858
ETAP [8]	E	0.705	0.598	0.888	0.883
MATE [9]	E	0.713	0.626	0.885	0.875

place relatively greater emphasis on later predictions and appear to benefit EDS, whereas larger values provide stronger performance on EC. Ultimately, the results indicate that the optimal weighting depends on the dataset, with $\gamma = 0.9$ providing the best overall trade-off and the strongest cross-dataset generalization.

Concluding our findings regarding the impact of the proposed loss, as shown in Table 1, synthetic and real-world performances show a notable discrepancy. Excluding RAFT-style loss yields a slight performance gain on synthetic data, reaching 0.845 FA, 0.835 EFA compared to 0.843 FA, 0.835 EFA when the loss is included. Conversely, on real datasets (EDS and EC), the RAFT-style loss consistently boosts both FA and EFA scores. This contrast reveals that training the model on synthetic data do not reliably predict real-world generalization; configurations that excel on synthetic data may fall short on real event-camera data.

EDS and EC Results. We evaluate our method using FA and EFA metrics, comparing its performance against classical feature tracking and both frame-based and frameless learning-based methods. The results we obtained are reported in Table 2.

It is clear that the proposed tracker delivers competitive results against existing event-based methods, particularly on the EC dataset and remains competitive on the EDS dataset, outperforming significantly the baseline DDFT, before and after finetuning. On the EDS dataset, the proposed pipeline improves over the learning-based baseline DDFT [19]. In particular, the proposed model achieves an FA of 0.616 and an EFA of up to 0.522, compared to 0.576 and 0.472 for DDFT. This indicates that the proposed pipeline, based on a Siamese FPN and a RAFT-style loss, improves tracking robustness with respect to the orig-

inal data-driven event-frame tracker. However, FE-TAP [18] remains the State of the Art method among the event-frame approaches on EDS, achieving 0.676 FA and 0.589 EFA. The event-only methods ETAP [8] and MATE [9] obtain the highest overall scores on this dataset, suggesting that EDS particularly benefits from the more recent event-only architectures. On the EC dataset, the proposed method performs strongly among event-frame trackers. With $\gamma = 0.9$, our method achieves the best event-frame performance, reaching 0.866 FA and 0.858 EFA. This improves upon FE-TAP, which obtains 0.844 FA and 0.838 EFA, and DDFT, which achieves 0.825 FA and 0.818 EFA. These results indicate that the proposed modifications improve generalization to real event-camera data and are particularly effective on EC. Although the proposed method remains slightly below the event-only ETAP and MATE trackers in absolute performance, the gap on EC is relatively small.

Concluding, the proposed pipeline substantially improves over DDFT on both real datasets and achieves the best event-frame performance on EC. Although the most recent event-only methods still obtain the highest overall scores, especially on EDS, our results show that the proposed Siamese event-frame tracker is a strong and robust alternative. The comparison at Table 1 further shows that synthetic improvements do not always transfer directly to real data. The RAFT-style loss is therefore selected not because it maximizes synthetic performance, but because it provides better real-data generalization, especially on EC.

6 Conclusion

In this paper a lightweight feature tracker for event cameras was proposed. Experimental results demonstrated that the proposed Siamese FPN architecture successfully bridges the gap between synthetic and real-world data, making the tracker highly adaptable in the domain shift problem. Moreover, although the RAFT-style sequence loss did not provide a clear gain on the synthetic MultiFlow test set, it consistently improved performance on the real EDS and EC datasets, indicating that supervising the full unrolled sequence was beneficial under real event-camera conditions. Compared with the original DDFT baseline, the proposed method achieved higher FA and EFA scores on both real benchmarks, with particularly strong performance on EC, where it outperforms the compared event-frame trackers. These results indicated that sharing the multi-scale encoder between the frame and event branches, together with sequence-level displacement supervision, yielded more robust feature tracking on real event-camera data. Given that recent event tracking pipelines increasingly rely on FPN-based extraction, the proposed Siamese FPN can replace disparate modality-specific encoders to achieve improved results. This issue is currently under investigation.

Acknowledgements

The authors gratefully acknowledge the Computer Centre of Computer Engineering & Informatics Dept. for providing the computational resources that supported this research.

References

1. Alzugaray, I., Chli, M.: HASTE: multi-hypothesis asynchronous speeded-up tracking of events. In: 31st British Machine Vision Conference 2020, BMVC 2020. BMVA Press (2020)
2. Bertinetto, L., Valmadre, J., Henriques, J.F., Vedaldi, A., Torr, P.H.S.: Fully-convolutional siamese networks for object tracking (2021)
3. Chui, J., Klenk, S., Cremers, D.: Event-based feature tracking in continuous time with sliding window optimization (2021)
4. Doersch, C., Yang, Y., Vecerik, M., Gokay, D., Gupta, A., Aytar, Y., Carreira, J., Zisserman, A.: Tapir: Tracking any point with per-frame initialization and temporal refinement (2023)
5. Fischer, P., Dosovitskiy, A., Ilg, E., Häusser, P., Hazırbaş, C., Golkov, V., van der Smagt, P., Cremers, D., Brox, T.: FlowNet: Learning optical flow with convolutional networks (2015)
6. Gehrig, D., Rebecq, H., Gallego, G., Scaramuzza, D.: Asynchronous, Photometric Feature Tracking Using Events and Frames, pp. 766–781. Springer International Publishing (2018). https://doi.org/10.1007/978-3-030-01258-8_46
7. Gehrig, M., Muglikar, M., Scaramuzza, D.: Dense continuous-time optical flow from event cameras. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **46**(7), 4736–4746 (2024). <https://doi.org/10.1109/tpami.2024.3361671>
8. Hamann, F., Gehrig, D., Febryanto, F., Daniilidis, K., Gallego, G.: Etap: Event-based tracking of any point (2025)
9. Han, H., Zhai, W., Cao, Y., Li, B., jun Zha, Z.: Mate: Motion-augmented temporal consistency for event-based point tracking (2025)
10. Harley, A.W., Fang, Z., Fragkiadaki, K.: Particle video revisited: Tracking through occlusions using point trajectories (2022)
11. Hidalgo-Carrio, J., Gallego, G., Scaramuzza, D.: Event-aided direct sparse odometry. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 5771–5780. IEEE (2022). <https://doi.org/10.1109/cvpr52688.2022.00569>
12. Karaev, N., Rocco, I., Graham, B., Neverova, N., Vedaldi, A., Rupprecht, C.: Cotracker: It is better to track together (2024)
13. Kueng, B., Mueggler, E., Gallego, G., Scaramuzza, D.: Low-latency visual odometry using event-based feature tracks. In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). pp. 16–23 (2016). <https://doi.org/10.1109/IROS.2016.7758089>
14. Li, B., Wu, W., Wang, Q., Zhang, F., Xing, J., Yan, J.: Siamrpn++: Evolution of siamese visual tracking with very deep networks (2018)
15. Li, B., Yan, J., Wu, W., Zhu, Z., Hu, X.: High performance visual tracking with siamese region proposal network. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (June 2018)

16. Li, S., Zhou, Z., Xue, Z., Li, Y., Du, S., Gao, Y.: 3d feature tracking via event camera. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 18974–18983 (June 2024)
17. Lin, T.Y., Dollar, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (July 2017)
18. Liu, J., Wang, B., Tan, Z., Zhang, J., Shen, H., Hu, D.: Tracking any point with frame-event fusion network at high frame rate (2024)
19. Messikommer, N., Fang, C., Gehrig, M., Cioffi, G., Scaramuzza, D.: Data-driven feature tracking for event cameras with and without frames. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **47**(5), 3706–3717 (May 2025). <https://doi.org/10.1109/tpami.2025.3536016>
20. Mueggler, E., Rebecq, H., Gallego, G., Delbruck, T., Scaramuzza, D.: The event-camera dataset and simulator: Event-based data for pose estimation, visual odometry, and slam. *The International Journal of Robotics Research* **36**(2), 142–149 (Feb 2017). <https://doi.org/10.1177/0278364917691115>
21. Seok, H., Lim, J.: Robust feature tracking in dvs event stream using bézier mapping. In: 2020 IEEE Winter Conference on Applications of Computer Vision (WACV). pp. 1647–1656 (2020). <https://doi.org/10.1109/WACV45572.2020.9093607>
22. Sun, D., Yang, X., Liu, M.Y., Kautz, J.: Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (June 2018)
23. Tao, R., Gavves, E., Smeulders, A.W.M.: Siamese instance search for tracking (2016)
24. Teed, Z., Deng, J.: Raft: Recurrent all-pairs field transforms for optical flow (2020)
25. Wang, Q., Zhang, L., Bertinetto, L., Hu, W., Torr, P.H.S.: Fast online object tracking and segmentation: A unifying approach (2019)
26. Zhu, A.Z., Atanasov, N., Daniilidis, K.: Event-based feature tracking with probabilistic data association. In: 2017 IEEE International Conference on Robotics and Automation (ICRA). pp. 4465–4470 (2017). <https://doi.org/10.1109/ICRA.2017.7989517>